

else

statements;

fi

The test-expression needs to return 0 (zero) to be *true* and any non-zero value to be *false*. In most languages, 1 (one) evaluates as *true* and 0 (zero) evaluates as *false*. Why does bash behave differently?

This is because shell scripts often need to determine how other programs have performed. They do this by reading the exit value of those programs. By convention, when a program exits without an error, it returns control to the invoking program with an exit code of 0 (zero). If it needs to exit after encountering an error, it returns with a non-zero exit code. To help in troubleshooting, program authors publish special meaning to each non-zero exit code.

Thus, in the *if* statement, the test-expression could be a program. If the program executed successfully and returned 0 to the shell, then the *if* statement behaves as if it evaluated to *true*. If the program exited with a non-zero value, then the *if* statement behaves as if it evaluated to *false*.

In Figure 1, the *if* statement evaluates commands and checks their exit values. It does not evaluate expressions as *true* or *false*.

What you need to remember is that the *if* statement is not looking for the boolean values — *true* or *false*.

Does this mean that *if true; then* it will evaluate to *false* because it is not 0 (zero)? No!

That brings us to another strange feature of the shell. *true* is actually a program! It is not part of the shell language. In Ubuntu, it resides in `/usr/bin/true` and exits with a return code of 0. There is also a *false* program residing at `/usr/bin/false`, which exits with a return code of 1.

```
~/Desktop
$ if rm sdfsd; then echo "Success"; else echo "Failure"; fi
rm: cannot remove 'sdfsd': No such file or directory
Failure

~/Desktop
$ if echo hello; then echo "Success"; else echo "Failure"; fi
hello
Success

~/Desktop
$
```

Figure 1: Exit code for *if* condition

```
~/Desktop
$ true; echo $?
0

~/Desktop
$ false; echo $?
1

~/Desktop
$ which true; which false; which [
/usr/bin/true
/usr/bin/false
/usr/bin/[

~/Desktop
$
```

Figure 2: What seem like key words or programming constructs are actually programs

[is a program

To test whether a file exists, you can use `if [ -f the-file.ext ]; then`. Here, the single bracket `[` is not part of the language. It is a program at `/usr/bin/[` and its arguments are: `-f, the-file.ext` and `]`.

To ensure that the `[` commands are executed properly, there has to be a space after the opening bracket and before the closing bracket. If you omit the first, you are not invoking the correct program. If you omit the latter, you have failed to terminate the command with the correct closing argument.

Beware of space in string comparisons

When you assign a value to a string variable, DO NOT leave any space before and after the `=` sign. If you do, it seems to the shell that the variable is a command and the `=` and the attempted value for the variable are its arguments.

When you test whether two strings are equal, DO leave a space before and after the `=` sign. If you do not, the `[` program will think you are trying to make an assignment. This assignment statement has an exit value of 0 (zero). This means the *if* statement will always be forced to evaluate to *true*!

```
# Causes an error because
'sTest' looks like a
command
# and '=' and '"hello"'
become its arguments
sTest = "hello"

# Assigns string variable
correctly
sTest="hello"
```

```
# Temporary assignment evaluates to
true whatever the value
if [ "$sTest"="hellooooooooo" ]; then
    echo "Yep"
else
    echo "Nope"
fi
```

```
# String comparison evaluates to true
if [ "$sTest" = "hello" ]; then
    echo "Yep"
else
    echo "Nope"
fi
```

// is not the fail-safe version of [

Unlike `[`, which is a program, the `[[` construct is a part of the shell language. Some misguided fellows on the internet recommend that you replace all your `[` evaluations with `[[` ones. Do not follow this advice.

`[[` is used for a more literal evaluation of text strings. You do not